1 EventHandler

1.1 System overview

Authorized subscriptions and event delivery. The native Go executable selects this system independently; no Java or Python process implements its operations. Shared source is in `internal/core`; federation code is in `internal/relay`.

1.2 Produced and consumed services

Produced HTTPS services: `echo`, `eventHandlerManagement`, `eventPublish`, `eventSubscription`, `generalManagement`. HTTPS service version 1.0.0. Optional native AMQPS telemetry produces `sysMonitor`; SysMonitor additionally publishes `sysMonitorData` over MQTT/TLS. Consumed authorities include ServiceRegistry, Authentication and ConsumerAuthorization; optional dependencies are explicit pinned HTTPS origins.

1.3 Important delimitations

This implementation is a development profile, not a certified interchangeable Java replacement. All systems use private JSON snapshots rather than the Java databases. MQTT application APIs, exhaustive DTO normalization and error parity, automatic data migration and production qualification remain outstanding.

- HTTPS callbacks use a persistent bounded outbox and at most three attempts. MQTT notifications, complete Java executor/backoff settings and distributed exactly-once guarantees are absent.

1.4 Security and persistence

Mutual TLS verifies the configured cloud trust anchors. Application identity tokens must match the client certificate. Authorities are pinned by system CN/profile before credentials are sent. Management requires a verified sysop identity or an explicit broad management whitelist. The default service authorization policy is enforce; identity-only mode must be explicitly selected. No insecure listener or declared-identity mode is implemented. Each executable owns one locked state file, written through fsync and atomic rename. Cross-system operations are not distributed transactions.

2 Implementation

Go 1.26 or newer builds the native binary. State is serialized as private JSON documents; updates clone state, validate, persist and then publish. Server timeouts and bounded message sizes constrain resource use. Persistent outboxes are at-least-once and cannot provide distributed exactly-once delivery. Detailed source, configuration and test instructions are in the repository README and verification report.

2.1 HTTPS interface operations

The listener is HTTPS with JSON by default; raw configuration and CRL responses use their declared media types. A request body is capped at 1 MiB. The table records implemented routes, not a claim of complete behavioral conformance.

Method	Path	Access
POST	/eventhandler/event-subscription/subscribe	identity
DELETE	/eventhandler/event-subscription/unsubscribe/{subscriptionId}	identity
GET	/eventhandler/management/subscriptions	management
PUT	/eventhandler/management/subscriptions/{subscriptionId}	management
DELETE	/eventhandler/management/subscriptions/{subscriptionId}	management
POST	/eventhandler/event-publish/publish	identity
GET	/eventhandler/monitor/echo	certificate
GET	/eventhandler/general/mgmt/get-config	management
POST	/eventhandler/general/mgmt/logs	management